

Universidad San Sebastian

Sistemas Operativos

Solemne 01 Practico - Parte 2 - Forma B

Informe final asincronico

Equipo 07 - Seccion NRC 18897

Lenguaje utilizado en la Parte 1: Python

Integrantes

Benjamin Zamora

Jose Palma

Franco Maripil

Nicolas Portilla

Thomas Marquez

09 de septiembre de 2026

1. Introduccion y objetivos

Este informe documenta el traslado de la solucion de monitoreo ambiental del Equipo 07 a un ambiente Debian GNU/Linux, como exige la Parte 2 de la Solemne 01 practica (Forma B): instalacion de la maquina virtual, preparacion del entorno, ejecucion de las dos versiones de la Parte 1 sobre el mismo conjunto de datos, gestor de incidencias y observacion de procesos, memoria y sistema de archivos. Todas las cifras se midieron dentro de la maquina virtual en una UNICA corrida y quedaron en evidencias/. El documento lo genera scripts/generar_informe.py, que lee esos archivos, los compara con el estado real del arbol entregado y se detiene sin emitir nada si ambas fuentes no coinciden.

1.1 Objetivo de cada componente

- src/secuencial.py: procesar un archivo JSONL completo antes de pasar al siguiente, sin hilos. Define el contrato de validacion y es la referencia contra la cual se compara el resultado concurrente.
- src/concurrente.py: procesar el mismo conjunto repartiendo los archivos entre 3 trabajadores threading.Thread mediante una queue.Queue compartida, protegiendo acumuladores globales y bitacora de alertas con mutex threading.Lock.
- src/gestor_incidencias.py: clasificar los informes por cantidad de alertas, resguardar el resumen, separar alertas por indicador, generar el inventario JSON y mantener una bitacora trazable sin detenerse ante entradas defectuosas.

1.2 Flujo del sistema de principio a fin

- Generacion de datos: scripts/generar_archivos_entrada.py crea 20 archivos estacion_CODIGO_AAAAMMDD.jsonl en entrada/ con semilla fija 20240908, de modo que la entrada es identica en cualquier maquina.
- Ingesta y validacion: cada linea JSONL se valida en formato, tipos, rangos y timestamp; la que no cumple se descarta como invalida sin detener el proceso.
- Procesamiento: la version secuencial recorre los archivos uno tras otro; la concurrente los encola en una queue.Queue y los reparte entre 3 hilos que acumulan resultados locales y solo actualizan las variables globales dentro de una seccion critica.
- Deteccion de alertas: cada medicion valida se contrasta con los umbrales y la alerta se escribe en alertas/alertas_detectadas.log con el formato archivo;estacion;timestamp;indicador;valor;umbral.
- Salida: un informe por archivo en salida/informe_*.txt y el consolidado salida/resumen_ambiental.txt.
- Gestion de incidencias: los informes se MUEVEN a gestion_ambiental/sin_alertas/, con_alertas/ o criticas/ segun sus alertas; el resumen se COPIA como resumen_resguardado.txt; las alertas se separan por indicador; se generan inventario_ambiental.json y logs/gestion_ambiental.log. Por eso salida/ queda solo con el resumen: ver 5.4.

1.3 Reglas de validacion y umbrales de alerta

Campo	Rango valido	Indicador	Condicion de alerta
Temperatura	-20.0 C a 60.0 C	Temperatura	mayor o igual a 35.0 C
Humedad	0.0 % a 100.0 %	Humedad	menor o igual a 20.0 %
PM2.5	mayor o igual a 0.0 ug/m3	PM2.5	mayor o igual a 35.0 ug/m3
Ruido	0.0 dB a 140.0 dB	Ruido	mayor o igual a 75.0 dB
Timestamp	formato AAAA-MM-DDTHH:MM:SS		

Ambas versiones comparten estas constantes, y esa es la razon de fondo por la cual sus metricas deben coincidir hasta el ultimo decimal: la concurrencia cambia el orden en que se hace el trabajo, no el criterio con que se decide.

2. Instalacion de Debian 13 en la maquina virtual

2.1 Descarga y verificacion del ISO

Se descargo la imagen de instalacion por red debian-13.6.0-amd64-netinst.iso (755 MB) desde el sitio oficial de Debian y se verifico su integridad comparando su SHA256 con el publicado en SHA256SUMS, lo que garantiza que el medio no fue alterado ni quedo truncado. La salida es la del anfitrión y esta archivada en evidencias/hyperv/configuracion-vm-hyperv.txt:

```
PS> Get-FileHash debian-13.6.0-amd64-netinst.iso -Algorithm SHA256
Hash calculado : 65273beed27b2df543b68b65630ba525cfbad8df2b12035732b2dff87d6664e7
Hash publicado : 65273beed27b2df543b68b65630ba525cfbad8df2b12035732b2dff87d6664e7
Fuente: SHA256SUMS
Coincide      : SI
```

2.2 Declaracion del cambio de hipervisor respecto del hito

Declaracion explicita. En el hito el equipo planifico usar Oracle VirtualBox y la implementacion final se hizo sobre Microsoft Hyper-V (maquina de Generacion 1, arranque BIOS). Se hace constar porque afecta lo declarado en el hito; la pauta admite la situacion, ya que la instalacion "puede hacerlo en VirtualBox, VMware u otro hipervisor". Los recursos comprometidos se respetaron exactamente, sin rebajar ninguno.

Recurso	Minimo de la pauta	Comprometido en el hito	Implementado y verificado
Hipervisor	VirtualBox, VMware u otro	Oracle VirtualBox	Microsoft Hyper-V, Generacion 1
RAM	2 GB	4 GB	4 GB fijos (memoria dinamica: False); 3.8Gi utiles segun free -h
Procesadores virtuales	2	4	4 vCPU sobre Intel(R) Core(TM) i9-14900
Disco virtual	20 GB	25 GB	25 GB Dynamic (ocupa 3 GB reales en el anfitrión)
Red	NAT	NAT con conectividad	Default Switch (Internal), IP 172.22.205.178/20 por DHCP
Sistema	Debian 13, 64 bits	Debian 13, 64 bits	Debian GNU/Linux 13 (trixie), version 13.6

El propio sistema instalado confirma sobre que hipervisor corre, de modo que la declaracion es verificable y no depende de la palabra del equipo:

```
$ systemd-detect-virt ; lscpu | grep 'Hypervisor vendor' ; uname -a
microsoft
Hypervisor vendor:                Microsoft
Linux debian-so-equipoo07 6.12.107+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.107-1 (2026-08-29) x86_64 GNU/Linux
```

La etapa previa sobre Oracle VirtualBox queda archivada en evidencias/fotos/instalacion/, donde consta la creacion de la maquina con esos mismos recursos. No se reproduce como figura porque no es la maquina entregada y sus valores ya estan verificados arriba contra los cmdlets del hipervisor.

2.3 Instalacion, usuario y particionado del disco virtual

Las capturas de esta seccion y de la siguiente son de la instalacion REAL de la maquina definitiva sobre Hyper-V. Se creo de Generacion 1, con firmware BIOS heredado, y su orden de arranque (IDE , CD , LegacyNetworkAdapter , Floppy) explica que el instalador arranque en modo BIOS y no UEFI. Se creo el usuario sin privilegios equipo07, con acceso a sudo, y el host quedo como debian-so-equipoo07, de modo que cualquier salida de este informe se atribuye sin ambigüedad a la maquina del equipo.

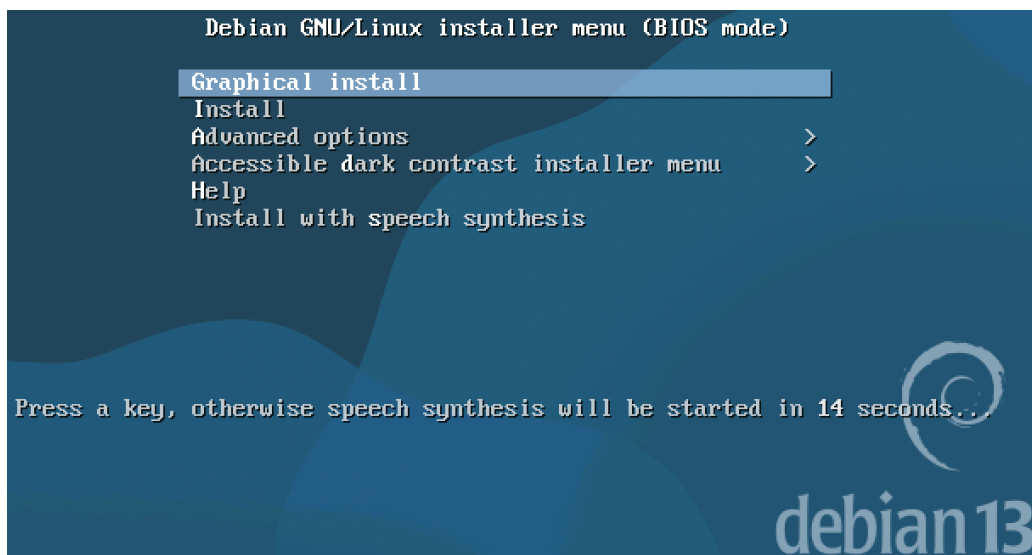


Figura 1. Menu del instalador de Debian 13 en modo BIOS, en la maquina de Generacion 1 de Hyper-V. Instalacion real de la maquina definitiva.

```
boot: install auto=true priority=critical interface=auto hostname=deb_
```

Figura 2. Linea de arranque del instalador con el archivo de preconfiguracion (preseed), que fija idioma, teclado, zona horaria, particionado guiado y el conjunto minimo de tareas. Usar preseed hace la instalacion reproducible: el mismo archivo produce la misma maquina.

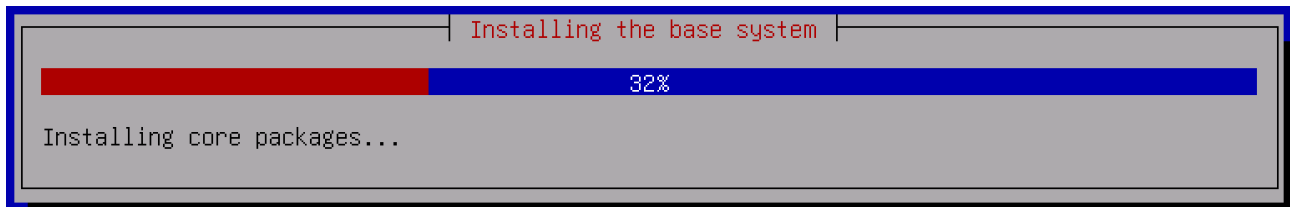


Figura 3. Instalacion del sistema base: descarga e instalacion de paquetes desde la replica de red. La etapa final del instalador -gestor de arranque y copia de la configuracion de red al sistema instalado- es la misma pantalla de progreso y esta archivada en evidencias/fotos/instalacion-hyperv/04-instalacion-final.png.

El particionado se aplico sobre el disco de 25 GB con el esquema guiado "todo en una particion":

```
$ lsblk
```

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINTS
fd0	2:0	1	4K	0	disk	
sda	8:0	0	25G	0	disk	
-sda1	8:1	0	23.7G	0	part	/
-sda2	8:2	0	1K	0	part	
-sda5	8:5	0	1.3G	0	part	[SWAP]
sr0	11:0	1	1024M	1	rom	

- sda1 (23.7 GB, ext4) es la particion raiz montada en /; sda2 es solo el contenedor logico de la extendida y sda5 (1.3 GB) es el area de intercambio, que el nucleo usa para descargar paginas cuando la memoria fisica escasea.
- sr0 es la unidad optica virtual desde la que se monto el ISO de instalacion; terminada la instalacion el ISO fue expulsado, como consta en la salida de Get-VMdvdDrive del anfitrión.

2.4 Primer inicio y conectividad

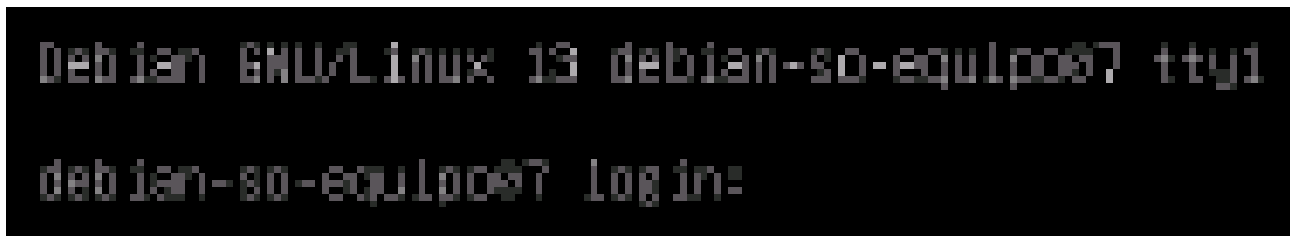


Figura 4. Primer inicio del sistema recién instalado: el núcleo llega a la consola tty1 y presenta el indicador de acceso del equipo debian-so-equip007. La captura muestra el indicador de acceso, no la sesión ya iniciada; que el acceso con el usuario equipo07 funciona lo demuestra el resto del informe, cuyas salidas fueron obtenidas en esa sesión.

eth0 obtiene su dirección por DHCP desde el conmutador Default Switch y la ruta por omisión apunta a 172.22.192.1. El Default Switch de Hyper-V provee NAT con DHCP hacia la red 172.22.192.0/20. El anfitrión ve la misma dirección (172.22.205.178, MAC 00155D647F07), lo que cierra la trazabilidad entre lo que declara Hyper-V y lo que observa el huésped. Con esa configuración la máquina alcanza las réplicas de Debian, como confirman la resolución de deb.debian.org y las líneas "Hit" de apt de la sección siguiente.

```
$ ip -4 -br addr show ; ip route ; getent hosts deb.debian.org
```

lo	UNKNOWN	127.0.0.1/8
eth0	UP	172.22.205.178/20
default via 172.22.192.1 dev eth0 proto dhcp src 172.22.205.178 metric 1002		
172.22.192.0/20 dev eth0 proto dhcp scope link src 172.22.205.178 metric 1002		
2a04:4e42:34::644 debian.map.fastlydns.net deb.debian.org		

3. Preparacion del ambiente

El punto 2.a de la pauta exige actualizar el sistema antes de trabajar. La salida confirma que los tres orígenes de paquetes están accesibles (trixie, trixie-security y trixie-updates) y que el sistema quedó al día.

```
$ sudo apt update ; sudo apt upgrade -y
```

```
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://security.debian.org/debian-security trixie-security InRelease
Hit:3 http://deb.debian.org/debian trixie-updates InRelease
All packages are up to date.

[...]
Calculating upgrade...
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

El punto 2.b pide instalar el entorno del lenguaje usado en la Parte 1. El proyecto está en Python y Debian 13 ya trae Python 3.13.5 en el sistema base, de modo que no hubo que compilar ni agregar repositorios externos. El proyecto usa exclusivamente la biblioteca estándar de Python: sin pip, sin entorno virtual y sin archivo de requisitos, decisión deliberada porque hace reproducible la ejecución incluso sin red. Los cuatro módulos compilan sin errores.

```
$ python3 --version ; apt list --installed | grep -E 'python3|openssh-server|time'
```

```
$ python3 -m py_compile src/*.py scripts/*.py # sin salida: los cuatro modulos compilan
```

```
Python 3.13.5  
openssh-server/stable,now 1:10.0p1-7+deb13u4 amd64 [installed]  
python3-venv/stable,now 3.13.5-1 amd64 [installed]  
python3/stable,now 3.13.5-1 amd64 [installed]  
time/stable,now 1.9-0.2 amd64 [installed]
```

4. Ejecucion de la Parte 1 en Debian

El punto 2.c exige ejecutar ambas versiones sobre el mismo conjunto de archivos .jsonl y comprobar que seis metricas coinciden. Para que "el mismo conjunto" sea verificable, la entrada se genera con semilla fija y se le calcula una suma de verificacion antes de cada corrida.

```
$ cd /home/equipo07/solemne_so_equipo07  
$ python3 scripts/generar_archivos_entrada.py  
$ cat entrada/*.jsonl | sha256sum  
$ time python3 src/secuencial.py  
$ time python3 src/concurrente.py
```

Archivos .jsonl generados: 20

```
d4f2b512dd71fd789fe9b2a3c1d0d69ee1bf3f4e8d7c99656dd113dad89b68f2 -
```

```
secuencial  real 0m0.019s  user 0m0.016s  sys 0m0.004s  (tiempo interno 0.005 s, 1 trabajador)  
concurrente real 0m0.023s  user 0m0.024s  sys 0m0.000s  (tiempo interno 0.007 s, 3 trabajadores)
```

4.1 Las seis metricas exigidas por la pauta

Metrica	Secuencial	Concurrente	Coincide
Archivos procesados	20	20	SI
Mediciones validas	300	300	SI
Mediciones invalidas	20	20	SI
Alertas totales	79	79	SI
PM2.5 maximo global	177.87 ug/m3	177.87 ug/m3	SI
Ruido maximo global	109.77 dB	109.77 dB	SI

Metricas comparadas: 6 de 6. TODAS LAS METRICAS EXIGIDAS COINCIDEN. El resumen consolidado agrega 320 lineas leidas, temperatura promedio 23.46 °C, humedad promedio 54.85 % y estacion con mas alertas STG04. El entregable conserva el resumen de la corrida concurrente en salida/resumen_ambiental.txt.

4.2 Comprobaciones adicionales de consistencia

Seis totales podrian coincidir por casualidad si dos errores se compensaran, de modo que se agregaron dos comprobaciones mas exigentes que las que pide la pauta: los 20 informes individuales comparados uno por uno, con 0 diferencias, y el contenido completo de la bitacora de alertas, con 79 alertas en el secuencial y 79 en el concurrente. El orden de escritura del concurrente varia por diseno, porque 3 hilos vuelcan sus buferes en momentos distintos; por eso el volcado final se ordena por nombre de archivo. IDENTICO BYTE A BYTE entre ambas versiones.

Resultado incomodo que se declara tal cual se midio: la concurrente fue mas LENTA que la secuencial en este conjunto (0m0.019s frente a 0m0.023s de tiempo real; 0.005 s frente a 0.007 s de tiempo interno). La explicacion esta en 8.1.

```
== Parte 1 en Debian: secuencial vs concurrente ==  
  
METRICA          SECUENCIAL      CONCURRENTE     OK  
-----  
Archivos procesados  20             20             SI  
Mediciones validas  300            300            SI  
Mediciones invalidas 20             20             SI  
Alertas totales     79             79             SI  
PM2.5 maximo global 177.87 ug/m3    177.87 ug/m3    SI  
Ruido maximo global 109.77 dB       109.77 dB       SI  
-----  
  
Informes individuales identicos entre versiones: 20 de 20  
alertas_detectadas.log: 79 lineas, identico byte a byte  
  
debian-so-equipo07 | Debian 13.6 | Python 3.13.5  
=
```

Figura 5. Comprobacion de consistencia en la consola de la maquina virtual: las seis metricas coinciden, los 20 informes individuales son identicos y la bitacora de alertas tiene 79 alertas en ambas versiones.

5. Gestor de incidencias

El punto 2.d enumera trece requisitos para src/gestor_incidentes.py. La tabla los recorre uno por uno con la evidencia que los respalda; todas las cifras provienen del estado real del proyecto y de los archivos de evidencia, y el generador comprueba que ambas fuentes coincidan antes de escribir esta pagina.

N	Requisito de la pauta	Evidencia obtenida
1	Detectar informes informe_CODIGO_AAAAMMDD.txt	Patron ^informe_[A-Za-z0-9]+_d{8}(_v\d+)?\.txt\$; 20 informes detectados
2	Leer la cantidad de alertas de cada informe	Linea "Alertas detectadas: N" de cada informe; suma 79 alertas
3	Mover informes con 0 alertas a sin_alertas/	Rama implementada; queda en 0 informes (ver 5.3) y se demostro con informe_CTL00_20240101.txt
4	Mover informes con 1 a 3 alertas a con_alertas/	8 informes en gestion_ambiental/con_alertas/
5	Mover informes con 4 o mas alertas a criticas/	12 informes en gestion_ambiental/criticas/
6	Evitar sobrescritura mediante nombres _v1, _v2	Tres corridas acumulan 61 informes, 40 de ellos con sufijo _v1 o _v2, sin perdidas (ver 7.3)
7	Copiar el resumen como resumen_resguardado.txt	Copia de 448 bytes, identica al original que permanece en salida/
8	Leer alertas/alertas_detectadas.log	79 lineas del formato archivo;estacion;timestamp;indicador;valor;umbral
9	Separar alertas por indicador	temperatura 21, humedad 17, pm25 20, ruido 21
10	Registrar alertas invalidas o desconocidas	4 anomalias registradas al inyectar fallas (ver 7.2)
11	Generar inventario_ambiental.json	Archivo de 3155 bytes, JSON valido con 8 claves de primer nivel
12	Generar logs/gestion_ambiental.log	Bitacora de 28 lineas y 2442 bytes, con marca de tiempo por evento
13	Controlar al menos un error sin detenerse	Con cuatro fallas el gestor termino con codigo 0 y conservo las 79 alertas (ver 7.2)

5.1 Estructura generada

```
$ find gestion_ambiental logs -type d | sort
gestion_ambiental
gestion_ambiental/alertas_por_indicador
gestion_ambiental/alertas_por_indicador/humedad
gestion_ambiental/alertas_por_indicador/pm25
gestion_ambiental/alertas_por_indicador/ruido
gestion_ambiental/alertas_por_indicador/temperatura
gestion_ambiental/con_alertas
gestion_ambiental/criticas
gestion_ambiental/sin_alertas
logs
```

Bajo gestion_ambiental/ quedaron 26 archivos: los 20 informes clasificados, los cuatro registros por indicador, el inventario y el resumen resguardado. El gestor termino con codigo 0 en su unica ejecucion sobre el arbol entregado.

```
== Estructura generada por el gestor de Incidencias ==

find gestion_ambiental -type d
gestion_ambiental
gestion_ambiental/alertas_por_indicador
gestion_ambiental/alertas_por_indicador/humedad
gestion_ambiental/alertas_por_indicador/pm25
gestion_ambiental/alertas_por_indicador/ruido
gestion_ambiental/alertas_por_indicador/temperatura
gestion_ambiental/con_alertas
gestion_ambiental/criticas
gestion_ambiental/sin_alertas

sin_alertas/      0 Informes
con_alertas/      8 Informes
criticas/         12 Informes

Alertas por indicador:
temperatura  21
humedad      17
pm25         20
ruido        21
SUMA         79 (= alertas_detectadas.log)

ls -lah gestion_ambiental/*.json gestion_ambiental/*.txt
-rw-rw-r-- 1 equipo07 equipo07 3.1K Sep  9 23:29 gestion_ambiental/inventario_ambiental.json
-rw-rw-r-- 1 equipo07 equipo07 448 Sep  9 23:29 gestion_ambiental/resumen_resguardado.txt
```

Figura 6. Estructura generada por el gestor y cuadratura de las alertas por indicador, en la maquina virtual.

5.2 Inventario y cuadratura de las alertas

El valor del inventario no esta en existir sino en cuadrar: la suma de las alertas por indicador debe ser igual al total del inventario, al numero de lineas de la bitacora de alertas y al total del resumen consolidado. El generador verifica esa igualdad y ademas que el numero leído en el repositorio sea el mismo de la evidencia congelada; si alguna comprobacion falla, se detiene.

Fuente	Alertas	Fuente	Alertas
alertas_por_indicador/temperatura	21	Suma de los cuatro indicadores	79
alertas_por_indicador/humedad	17	alertas/alertas_detectadas.log	79
alertas_por_indicador/pm25	20	inventario_ambiental.json	79
alertas_por_indicador/ruido	21	salida/resumen_ambiental.txt	79

```
$ python3 -m json.tool gestion_ambiental/inventario_ambiental.json
```

```
{
  "total_informes": 20,
  "informes_por_categoria": {
    "sin_alertas": 0,
    "con_alertas": 8,
    "criticas": 12
  },
  "total_alertas": 79,
  "anomalias_registradas": 0,
  "errores_de_proceso": 0
}
...
[2026-09-09 23:29:11] Inventario generado en gestion_ambiental/inventario_ambiental.json: 20 informes, 79 alertas.
[2026-09-09 23:29:11] CONTROL DE ERRORES: no se detectaron anomalias en esta corrida. El procesamiento termino completo.
[2026-09-09 23:29:11] --- FIN DE GESTION DE INCIDENCIAS ---
```

5.3 Por que sin_alertas/ quedo vacia

La carpeta sin_alertas/ contiene 0 informes y no es un defecto del gestor: la pauta de la Parte 1 exige que cada archivo de entrada produzca al menos una alerta, de modo que ningun informe puede tener cero y la rama queda legitimamente vacia. La rama existe y funciona: con el informe de control informe_CTL00_20240101.txt, construido con cero alertas, el gestor lo movio a sin_alertas/ y la clasificacion quedo en sin_alertas/ 1, con_alertas/ 8, criticas/ 12. Esa demostracion se hizo sobre una COPIA, por lo que el arbol entregado conserva sus 0 informes en esa rama.

5.4 Por que salida/ solo conserva el resumen

En el arbol entregado, salida/ contiene solo resumen_ambiental.txt y LEEME.txt. No falta nada: los 20 informes individuales SI se generaron ahi (src/concurrente.py) y el gestor los MOVIO a sus carpetas de clasificacion, que es lo que exigen los puntos 3, 4 y 5 de la pauta. Mover y no copiar se explica en 7.1. Antes de ejecutar el gestor se respaldo salida/ completo, de modo que el estado que dejo la Parte 1 tambien es auditable:

- salida/resumen_ambiental.txt (448 bytes): el consolidado que la pauta pide mantener en su lugar.
- salida/LEEME.txt: explica esta situacion dentro del propio entregable e indica como regenerar los informes.
- evidencias/salida_parte1/: 21 archivos, es decir los 20 informes individuales mas el resumen, tal como los dejo la Parte 1 antes de la clasificacion.

6. Observacion de procesos y del sistema de archivos

Nota metodologica sobre la carga amplificada. El conjunto oficial de 20 archivos se procesa en milisegundos y ps no alcanza a tomar una muestra util. Por eso la observacion se hizo en dos partes: (A) el consumo de la corrida OFICIAL con /usr/bin/time -v, que no necesita muestreo; y (B) la observacion en vivo con ps sobre una CARGA AMPLIFICADA de 4000 archivos (64000 lineas), los mismos archivos oficiales replicados con otras fechas validas. Ambas partes se ejecutan sobre COPIAS del proyecto, para no alterar el arbol entregable. Toda metrica entregable sale del conjunto oficial; la carga amplificada solo hace que el proceso viva lo suficiente para observarlo (270 muestras).

6.1 Consumo de recursos de la corrida oficial

```
$ /usr/bin/time -v python3 src/concurrente.py
```

```
Command being timed: "python3 src/concurrente.py"
User time (seconds): 0.01
System time (seconds): 0.00
Percent of CPU this job got: 100%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.02
Maximum resident set size (kbytes): 12964
Voluntary context switches: 149
Involuntary context switches: 1
File system inputs: 0
File system outputs: 184
```

- Percent of CPU 100% sobre centesimas de segundo: con solo 20 archivos el proceso apenas alcanza a repartir trabajo antes de terminar.
- Maximum resident set size 12964 KB (unos 12.7 MB de memoria fisica): el costo del interprete de Python mas los acumuladores del programa.
- 149 cambios de contexto voluntarios y 1 involuntarios. Los voluntarios son el proceso cediendo la CPU por su cuenta al esperar entrada/salida o un mutex; los involuntarios son el planificador expulsandolo al agotar su cuanto. 184 bloques escritos corresponden a los informes, el resumen y la bitacora de alertas.

6.2 Identificacion del proceso concurrente

```
$ ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,nlwp,etime,cmd -p <PID>
```

```
PID    PPID  STAT   %CPU  %MEM    RSS     VSZ  NLWP   ELAPSED CMD
10043  10018 Sl      100   0.4  19368  249776  4      00:00 python3 src/concurrente.py
```

Campo	Valor	Que significa en este proceso concreto
PID	10043	Identificador que el nucleo asigno al proceso que ejecuta python3 src/concurrente.py; es unico mientras el proceso vive.
PPID	10018	Proceso padre: 10018 Ss bash /tmp/rsh_13-procesos-recaptura.sh. Es el shell desde el que se lanzo la ejecucion.
STAT	Sl	S: en el instante de la muestra el proceso dormia en espera interrumpible (entrada/salida o mutex); la l final significa multihilo. En otras muestras aparece R, ejecutandose en CPU.
%CPU	100 (max. 103)	Que supere el 100 % es la prueba de trabajo simultaneo en mas de un nucleo: un proceso de un solo hilo no puede pasar de 100 %.
%MEM	0.4	Fraccion de los 4 GB de memoria fisica ocupada por el proceso; es baja porque el programa procesa por flujo y no carga todo en memoria.
RSS	19368 KB	Memoria residente: la parte del proceso efectivamente cargada en RAM, que es lo que cuesta de verdad en memoria fisica.
VSZ	249776 KB	Memoria virtual reservada, unas trece veces el RSS: incluye bibliotecas compartidas, pilas de hilos y regiones que nunca se tocaron, por lo que no debe leerse como consumo real.
NLWP	4	Hilos vivos: 1 principal + 3 trabajadores, exactamente los que declara el programa.

6.3 Hilos y vista del nucleo

```
$ ps -L -o pid,tid,stat,%cpu,comm -p 10043
$ grep -E 'State|Threads|VmRSS' /proc/10043/status
```

```
  PID      TID  STAT  %CPU  COMMAND
  10043    10043 Sl    75.0  python3
  10043    10108 Sl     0.0  python3
  10043    10109 Sl     0.0  python3
  10043    10111 Rl     0.0  python3
Name:      python3
State:     S (sleeping)
Tgid:      10043
Pid:       10043
PPid:      10018
VmSize:    249776 kB
VmRSS:     18908 kB
Threads:   4
```

Los cuatro identificadores de hilo comparten el mismo PID pero tienen TID distintos: el primero coincide con el PID y es el hilo principal, que encola y espera; los otros 3 consumen de la cola. El nucleo confirma Threads: 4 y VmRSS: 18908 kB, coherente con ps, y State: S (sleeping). Que el estado sea "sleeping" en una muestra y "running" en otra no es contradictorio: los hilos alternan entre calculo y espera por entrada/salida.

```
$ ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,nlwp,cmd -p <PID> # muestras sucesivas
10043  10018 R    0.0  0.1  7036  19344  1      00:00 python3 src/concurrente.py
10043  10018 Sl   100  0.4  19368  249776  4      00:00 python3 src/concurrente.py
10043  10018 Sl   102  0.4  19964  250800  4      00:00 python3 src/concurrente.py
10043  10018 Sl   103  0.5  20876  251824  4      00:00 python3 src/concurrente.py
10043  10018 R    103  0.5  21600  251776  1      00:00 python3 src/concurrente.py
```

La primera muestra fue tomada antes de que arrancaran los trabajadores: un solo hilo, 0.0 % de CPU y unos 7 MB residentes. En las siguientes NLWP sube a 4, el %CPU pasa de 100 y el RSS crece de forma sostenida a medida que se llenan los acumuladores. Esa progresion es la concurrencia hecha visible.


```

== Observacion del proceso concurrente ==
Carga amplificada: 4000 archivos, 64000 lineas (16 estaciones x 250 fechas)

ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,nlwp,cmd -p PID
  PID  PPID  STAT  %CPU  %MEM   RSS   VSZ  NLWP  CMD
  12199  12187  Sl      107   8.5 20400 250B08    4 python3 src/concurrente.py

%CPU maximo observado con los 4 hilos vivos: 107
(por encima de 100 = trabajo simultaneo en varios nucleos)

Hilos vivos (ps -L): 1 principal + 3 trabajadores
  PID  TID  STAT  COMMAND
  12199  12199  Sl    python3
  12199  12253  Rl    python3
  12199  12254  Rl    python3
  12199  12255  Rl    python3
State:  S (sleeping)
Pid:    12199
PPid:   12187
VmRSS:  18540 kB
Threads: 4

du -sh ~/solemne_so_equipo07 : df -h /
624K    /home/equipo07/solemne_so_equipo07
/dev/sda1    24G  1.2G  21G   6% /
=

```

Figura 7. Observacion en vivo del proceso concurrente en la consola de la maquina virtual. Es otra corrida de la misma prueba, con su propia carga amplificada -la que declara el encabezado de la pantalla, distinta de la transcrita arriba- y por eso muestra otro PID y otro RSS; coincide en estructura y en orden de magnitud: un proceso con 4 hilos vivos, %CPU por encima de 100 y el mismo du -sh de 624K del proyecto.

6.4 Memoria del sistema, espacio disponible y tamano del proyecto

```

$ free -h ; df -h / ; du -sh ~/solemne_so_equipo07

```

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	379Mi	3.3Gi	3.7Mi	373Mi	3.5Gi
Swap:	1.3Gi	0B	1.3Gi			

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	24G	1.2G	21G	6%	/


```

624K    /home/equipo07/solemne_so_equipo07

```

La maquina tiene 3.8Gi de memoria total, con 379Mi en uso y 3.3Gi libres. La columna buff/cache (373Mi) no es memoria perdida sino cache de disco que el nucleo devuelve en cuanto un proceso la necesita, y por eso la disponible (3.5Gi) supera a la libre; el intercambio queda en 0 B usados, senal de que el trabajo nunca presiono la memoria fisica. El proyecto vive en /dev/sda1 montado en /, con 24G de capacidad, 1.2G usados y 21G disponibles (6% de uso), y ocupa 624K; ese total supera la suma de sus subcarpetas por el redondeo de du a bloques de 4096 bytes.

6.5 Metadatos de la bitacora con stat

```

$ stat logs/gestion_ambiental.log ; ls -lah logs/gestion_ambiental.log

```

```

File: logs/gestion_ambiental.log
Size: 2442      Blocks: 8      IO Block: 4096   regular file
Device: 8,1    Inode: 1046636 Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/equipo07)   Gid: ( 1000/equipo07)
Access: 2026-09-09 23:29:11.762321294 -0300
Modify: 2026-09-09 23:29:11.766321401 -0300
Change: 2026-09-09 23:29:11.766321401 -0300
 Birth: 2026-09-09 23:29:11.762321294 -0300
-rw-rw-r-- 1 equipo07 equipo07 2.4K Sep  9 23:29 logs/gestion_ambiental.log

```

Campo de stat	Valor	Lectura
Tipo y tamano	regular file, 2442 B en 8 bloques	Archivo ordinario de datos. El tamano logico es menor que el espacio reservado, porque el sistema de archivos asigna bloques completos de 4096 bytes.
Inodo y enlaces	1046636 (dispositivo 8,1), 1 enlace	El nombre gestion_ambiental.log es solo una entrada de directorio que apunta a ese inodo. Un enlace duro subiria el contador a 2 sin duplicar un byte.
Permisos	0664/-rw-rw-r--	Lectura y escritura para el propietario y su grupo, solo lectura para el resto: la bitacora es auditable por terceros pero no modificable.

Campo de stat	Valor	Lectura
Propietario / grupo	1000/equipo07 / 1000/equipo07	Pertenece al usuario sin privilegios del equipo, no a root: el gestor no necesita permisos elevados para operar.
Access / Modify / Change / Birth	iguales al segundo	Ultima lectura, ultima escritura de contenido, ultimo cambio de metadatos del inodo y creacion: coinciden porque el archivo se creo y se escribio en la misma corrida.

Esa bitacora tiene 28 lineas y 2442 bytes. Ambas cifras corresponden a la misma corrida: el generador de este informe compara el conteo de lineas del archivo entregado con el que declara la evidencia, y el tamaño del archivo entregado con el que devolvió stat, y se detiene si alguno de los dos pares no calza.

7. Mover frente a copiar, y control de errores

7.1 Por que los informes se mueven y el resumen se copia

Los informes se MUEVEN desde salida/ a su carpeta de clasificación porque el traslado es un cambio de estado definitivo: un informe ya clasificado no debe seguir figurando como pendiente. Copiarlos duplicaría cada informe y se perdería la propiedad más útil: que salida/ sin informes significa "no queda nada por clasificar".

Mover dentro de una misma partición (aquí origen y destino están ambos en /dev/sda1) es la llamada rename(2): no copia datos, elimina la entrada de directorio antigua y crea otra que apunta al MISMO inodo. Los bloques no se tocan, el número de inodo no cambia y solo se actualiza el ctime; por eso es casi instantánea y su costo no depende del tamaño. Entre particiones distintas el núcleo no podría renombrar y shutil.move degradaría a copiar y borrar.

El resumen consolidado tiene otro papel: es la vista general que el módulo de monitoreo consulta en su ruta de siempre. Por eso salida/resumen_ambiental.txt se CONSERVA y además se COPIA como gestion_ambiental/resumen_resguardado.txt con shutil.copy. Copiar si crea un inodo nuevo, con su contenido en otros bloques: desde ese momento los dos archivos son independientes y modificar uno no altera al otro, que es lo que se espera de un respaldo.

	Mover (rename)	Copiar (copy)
Se aplica a	informe_*.txt	resumen_ambiental.txt
Efecto en el inodo	conserva el mismo inodo	crea un inodo nuevo
Datos en disco	no se duplican	se duplican
Original	deja de existir en el origen	permanece en salida/
Propósito	reclasificación definitiva	respaldo y auditoría

Comprobación posterior a la ejecución del gestor

```
salida/resumen_ambiental.txt sigue existiendo : SI
identico a resumen_resguardado.txt           : SI
informes que quedan en salida/                 : 0 (se MOVIERON)
```

El resumen original (448 bytes) sigue en salida/, la copia resguardada pesa 448 bytes y su contenido es idéntico byte a byte: comprobado. En salida/ quedan 0 informes individuales, porque todos fueron movidos y no copiados; esto es lo que documenta 5.4.

7.2 Control de errores sin detener el procesamiento

Esta demostración es DESTRUCTIVA: inyecta datos dañados. Por eso NO se ejecuto sobre el árbol entregado sino sobre una COPIA completa en /home/equipo07/demo_destructiva. De ahí que la bitacora entregada cierre con "no se detectaron anomalías en esta corrida": el entregable viene de una corrida limpia y las cifras de esta subsección pertenecen a la copia.

El requisito 13 pide controlar al menos un error sin detener el procesamiento. Se inyectaron cuatro fallas distintas, para probar tanto el análisis de la bitacora de alertas como la lectura de los informes: una línea sin separadores, otra con campos insuficientes, un indicador desconocido ("Radiación") y un informe sin la línea "Alertas detectadas: N".

```
$ python3 src/gestor_incidentes.py ; echo "Codigo de salida: $?"
$ grep 'ANOMALIA|CONTROL DE ERRORES' logs/gestion_ambiental.log
Codigo de salida: 0
```

```
[2026-09-09 23:29:11] ANOMALIA: el informe informe_XXX99_20240101.txt no declara la línea 'Alertas detectadas: N'. No se
clasifica, queda en salida/ para revisión manual. Se continúa.
[2026-09-09 23:29:11] ANOMALIA: alerta con formato inválido en la línea 80 de alertas/alertas_detectadas.log:
'línea_corrupta_sin_separadores'. Se descarta y se continúa.
[2026-09-09 23:29:11] ANOMALIA: alerta con formato inválido en la línea 81 de alertas/alertas_detectadas.log:
'faltan;campos;aquí'. Se descarta y se continúa.
[2026-09-09 23:29:11] ANOMALIA: indicador desconocido 'Radiación' en la línea 82 de alertas/alertas_detectadas.log. Se
descarta y se continúa.
```

```
CONTROL DE ERRORES: se detectaron 4 anomalias (2 alertas con formato invalido, 1 con indicador desconocido, 0 lineas con bytes corruptos, 1 informes sin dato de alertas, 0 archivos ignorados en salida, 0 advertencias sobre el origen de alertas, 0 errores de proceso capturados). Todas quedaron registradas y el procesamiento CONTINUO hasta generar el inventario.
```

El comportamiento es el correcto: las 4 anomalias quedaron registradas con su numero de linea y su contenido, el gestor NO se detuvo, termino con codigo 0, conservo las 79 alertas validas y genero el inventario igual. Descartar el dato defectuoso y continuar es preferible a abortar: una linea corrupta no puede invalidar una jornada de monitoreo, pero tampoco puede desaparecer en silencio.

```
== stat de la bitacora del entregable ==

  File: logs/gestion_ambiental.log
  Size: 2442      Blocks: 8      IO Block: 4096   regular file
Device: 8,1  Inode: 1046636   Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/equipo07)   Gid: ( 1000/equipo07)
Access: 2026-09-09 23:29:11.806322470 -0300
Modify: 2026-09-09 23:29:11.766321401 -0300
Change: 2026-09-09 23:29:11.766321401 -0300
Birth:  2026-09-09 23:29:11.762321294 -0300

== Control de errores (sobre una copia, para no alterar el entregable) ==
Se inyectaron 2 alertas danadas en alertas_detectadas.log
Codigo de salida del gestor : 0   (no se detuvo)
Anomalias registradas       : 2
Alertas conservadas        : 79

[2026-09-09 23:30:38] CONTROL DE ERRORES: se detectaron 2 anomalias (1 alertas
```

Figura 8. Metadatos de la bitacora con stat y evidencia del control de errores en la consola de la maquina virtual. La captura corresponde a una corrida de control anterior, con dos alertas danadas en vez de las cuatro fallas transcritas arriba; por eso declara dos anomalias y otro tamaño de bitacora. El resultado es el mismo: registra cada anomalía, no se detiene, termina con código 0 y completa el inventario con las 79 alertas. La última línea queda cortada por el borde de la consola; su texto completo esta en evidencias/debian/06-control-de-errores.txt.

7.3 Idempotencia y proteccion contra sobrescritura

Esta demostracion tambien es destructiva, porque duplica informes a proposito, y por eso se ejecuto sobre la misma COPIA /home/equipo07/demo_destructiva. El arbol entregado conserva sus 20 informes sin sufijos de version, que es el estado de una unica corrida limpia.

```
# Tres corridas consecutivas del gestor sobre la copia
$ ls -lah gestion_ambiental/criticas/informe_STG04_20240101*

corrida 1 -> alertas por indicador: 79 | inventario: 79
corrida 2 -> alertas por indicador: 79 | inventario: 79
corrida 3 -> alertas por indicador: 79 | inventario: 79

-rw-rw-r-- 1 equipo07 equipo07 364 Sep  9 23:29 informe_STG04_20240101.txt
-rw-rw-r-- 1 equipo07 equipo07 364 Sep  9 23:29 informe_STG04_20240101_v1.txt
-rw-rw-r-- 1 equipo07 equipo07 364 Sep  9 23:29 informe_STG04_20240101_v2.txt
```

Las alertas no se duplican entre corridas porque los registros por indicador se reconstruyen de forma atomica sobre archivos temporales que se renombran al final. Al reprocesar, los informes ya clasificados no se pierden ni se pisan: obtener_nombre_seguro() comprueba la existencia del destino y agrega un sufijo correlativo. Partiendo de 21 informes, tras dos ciclos adicionales conviven 61 donde se esperaban 61 (40 con sufijo): ningun archivo fue sobrescrito ni eliminado.

8. Conclusiones

8.1 Sobre concurrencia: el resultado que no favorece la hipotesis

La conclusion mas importante es tambien la mas incomoda: en este caso concreto la concurrencia NO acelero el procesamiento. La version concurrente tardo 0m0.023s frente a 0m0.019s de la secuencial, y su tiempo interno fue 0.007 s frente a 0.005 s. El dato no se maquilla, se explica:

- El trabajo util es demasiado pequeno: 20 archivos con 320 lineas, procesados en centesimas de segundo.
- El costo fijo de la concurrencia no depende del tamaño del trabajo: crear 3 hilos, encolar los archivos y tomar y soltar los mutex en cada actualizacion pesa mas que lo que se ahorra al repartir un trabajo tan breve.
- El trabajo esta dominado por entrada/salida sobre archivos diminutos que el nucleo ya tiene en cache, de modo que hay poca espera que solapar; y en CPython el bloqueo global del interprete impide que dos hilos ejecuten codigo Python puro a la vez.

La concurrencia si se observa cuando el trabajo crece: con la carga amplificada de 4000 archivos el proceso alcanzo 103 % de CPU, por encima del 100 % que jamas superaria un proceso de un solo hilo. El mecanismo funciona; lo que falta en el conjunto oficial es trabajo suficiente para amortizar su costo de entrada.

8.2 Sobre sincronizacion y sistema de archivos

Que las 6 de 6 metricas coincidan y que los 20 informes sean identicos es la evidencia de que la sincronizacion es correcta: con 3 hilos actualizando contadores compartidos, sin mutex cada corrida habria dado un resultado distinto. El patron fue acumular en variables locales y entrar a la seccion critica una sola vez por archivo; queue.Queue garantiza que cada archivo lo toma un unico trabajador. El unico efecto observable de la concurrencia es el ORDEN de las lineas de la bitacora.

Mover y copiar obliga a distinguir el nombre de un archivo del archivo mismo: un nombre es una entrada de directorio que apunta a un inodo. Las herramientas completaron el cuadro: stat dio inodo, enlaces, permisos y marcas de tiempo; ls -lah y du -sh, la diferencia entre tamano logico y bloques ocupados; df -h, el sistema de archivos y su espacio libre; free -h, que la cache no es memoria perdida; y ps con /proc, la estructura interna del proceso.

8.3 Declaraciones de honestidad tecnica

- Hipervisor: el hito declaro Oracle VirtualBox y la implementacion final se hizo sobre Microsoft Hyper-V (maquina de Generacion 1, arranque BIOS), con los recursos comprometidos intactos; la pauta admite otro hipervisor. TODAS las figuras de instalacion son de la maquina real en Hyper-V; la etapa previa sobre Oracle VirtualBox queda archivada, no presentada como evidencia de la maquina entregada.
- Rendimiento: la version concurrente resulto mas lenta que la secuencial en el conjunto oficial; se informa tal como se midio.
- Clasificacion: sin_alertas/ quedo con 0 informes porque la pauta exige al menos una alerta por archivo; la rama se demostro aparte.
- Demostraciones destructivas: el control de errores (7.2) y la anti-sobrescritura (7.3) se ejecutaron sobre una COPIA, no sobre el arbol entregado; por eso la bitacora entregada no registra esas anomalias.
- Trazabilidad: cada cifra se lee al generar el informe desde evidencias/ y del estado real del proyecto, y el generador compara ambas fuentes -y las cifras del README- antes de escribir nada; si discrepan, no se emite el documento.

8.4 Indice de evidencias que respaldan este informe

Cada seccion puede auditarse contra el archivo que la origina; el generador falla si alguno cambia de forma incompatible.

Archivo de evidencia	Contenido	Secciones
evidencias/hyperv/configuracion-vm-hyperv.txt	cmdlets de Hyper-V y verificacion SHA256 del ISO	2.1, 2.2
evidencias/debian/01-preparar-ambiente.txt	apt update, apt upgrade y entorno de Python	3
evidencias/debian/02-entorno.txt	distribucion, kernel, hipervisor, CPU, memoria, disco y red	2
evidencias/debian/03-ejecucion-parte1.txt	ambas corridas, las seis metricas y la comparacion de informes y alertas	4
evidencias/debian/04-observacion-procesos.txt	/usr/bin/time -v, ps, ps -L, /proc, free -h, df -h y du -sh	6
evidencias/debian/05-gestor-incidencias.txt	estructura, inventario, stat de la bitacora, mover frente a copiar	5, 7.1
evidencias/debian/06-control-de-errores.txt	anomalias inyectadas y rama sin_alertas (sobre una copia)	5.3, 7.2
evidencias/debian/07-anti-sobrescritura.txt	sufijos _v1 y _v2 e idempotencia (sobre una copia)	7.3
evidencias/salida_parte1/	21 archivos: salida/ tal como la dejo la Parte 1	5.4
evidencias/fotos/	instalacion real en Hyper-V y consola de la maquina definitiva	Figuras 1 a 8
gestion_ambiental/, alertas/, logs/, salida/	estado real: informes, alertas por indicador, inventario y bitacora	5, 7
docs/Solemne01PracticaParte2FormaB.pdf	pauta oficial contra la cual se verifico cada requisito	todas